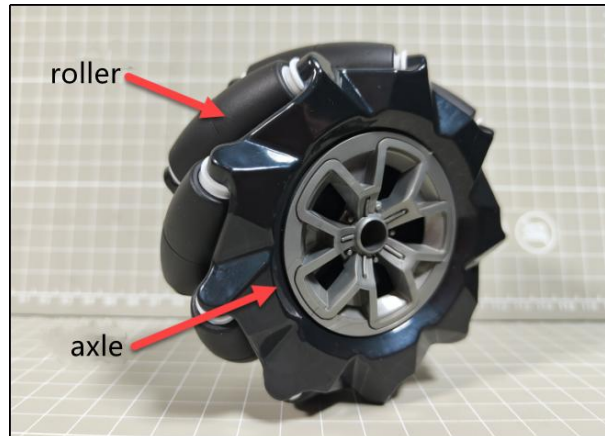


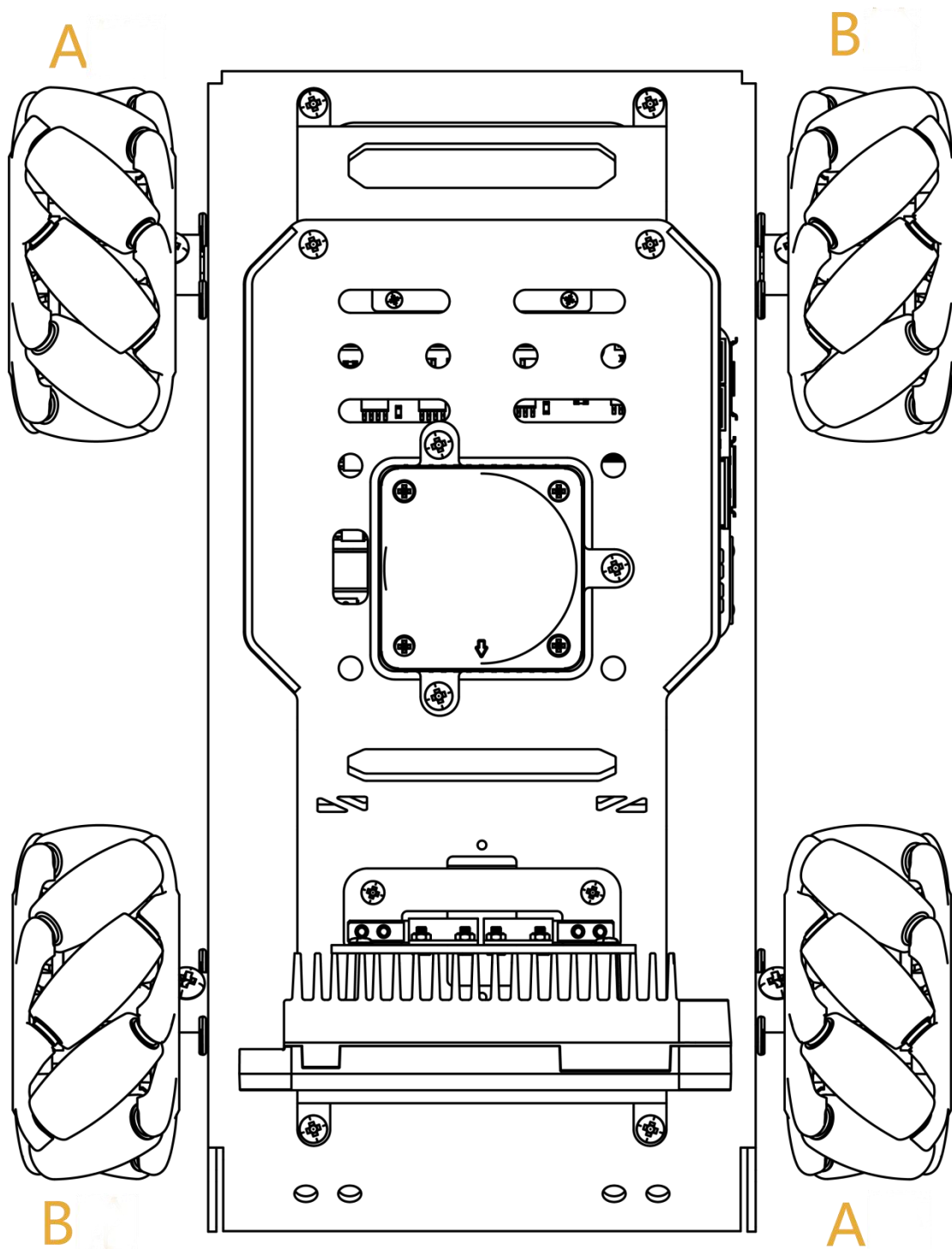
Mecanum Wheel Chassis Motion Analysis

1. Hardware Structure



The Mecanum wheel consists of rollers mounted on an axle, which serves as the main support structure. The axle is positioned at a 45-degree angle to the roller axis. Typically, Mecanum wheels are used in sets of four: two left wheels and two right wheels, with Wheels A and B being symmetrical.

There are several possible combinations for the four Mecanum wheels, including AAAA, BBBB, AABB, ABAB, and BABA. However, not all combinations enable the robot to move in all directions—forward, backward, and sideways. The ABAB combination of Mecanum wheels is specifically designed for omnidirectional movement.



2. Physical Characteristics

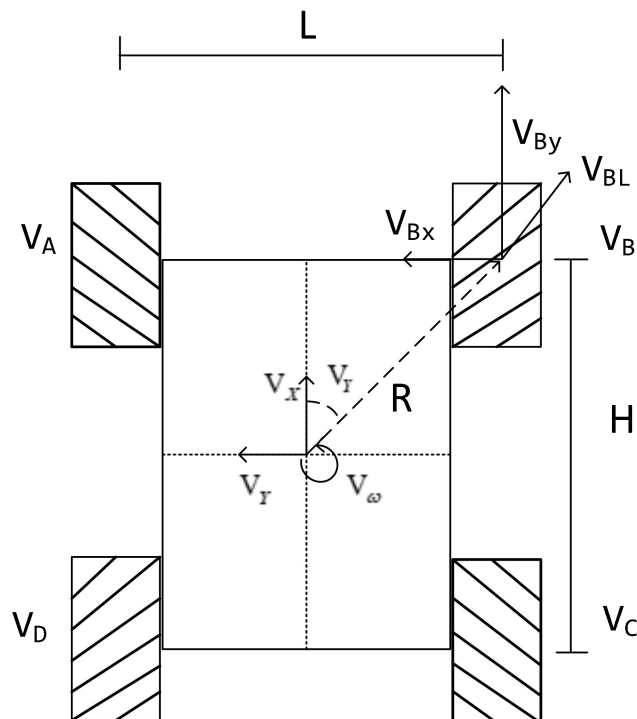
The vehicle achieves omnidirectional movement by combining the propulsion

forces from the ground-engaging rollers. This force combination can be directed in any direction by adjusting the rotation direction and torque of the four wheels.

Due to the specific orientation of the rollers at a precise angle to the wheel circumference, Mecanum wheels are capable of sliding sideways. The generatrix of these small rollers is distinct, and as the Mecanum wheel rotates around its fixed axle, the envelope of each roller forms a cylindrical surface, enabling continuous forward motion.

3. Motion Principle and Formula

When conducting kinematic analysis, we can consider the kinematic model of Mecanum wheels, which includes the following parameters:



1. V_x : Velocity of the Mecanum wheel in the X-axis (typically front and rear direction).
2. V_y : Velocity of the Mecanum wheel in the Y-axis (typically left and right

direction).

3. V_{ω} : Angular velocity of the Mecanum wheel chassis (rotation speed of the chassis around its own center).
4. Real-time velocities of the four wheels of the Mecanum wheel.
5. The motion of the right front wheel in the plane can be decomposed into:
6. V_{Bx} : Velocity of the Mecanum wheel in the X-axis (typically front and rear direction).
7. V_{By} : Velocity of the Mecanum wheel in the Y-axis (typically left and right direction).
8. L : Distance between the centers of the left and right wheels.
9. H : Distance between the centers of the front and rear wheels.
10. θ_{ω} : Angle formed by the chassis body center and the center of the right front wheel, typically 45° .
11. With these parameters, we can perform kinematic analysis of the Mecanum wheel chassis. The following are key mathematical formulas:

Kinematics Formula:

To simplify the mathematical model for kinematics, we make the following two idealized assumptions:

- 1) Omni-directional wheels do not slip on the ground, and there is sufficient friction with the ground.
- 2) The 4 wheels are distributed at the corners of a rectangle or square, with the wheels parallel to each other.

Here, we decompose the rigid body motion of the car into three components linearly. By calculating the velocities of the four wheels when the output Mecanum wheel chassis translates along the X+ and Y+ directions and rotates along the Z+ direction, we can combine these three simple motions using

formulas to determine the required speeds of the four wheels.

In the equations, A, B, C, and D represent the rotational speeds of the four wheels, i.e., the motor speeds. V_X is the translation speed of the car along the X-axis, V_Y is the translation speed along the Y-axis, and ω is the rotational speed along the Z-axis. $L/2$ is half of the car's wheelbase, and $H/2$ is half of the car's axle distance.

1. The velocity components of each wheel during the robot's translation along the X-axis can be calculated using the following formulas:

$$V_A = +V_X \quad , \quad V_B = +V_X \quad , \quad V_C = +V_X \quad , \quad V_D = +V_X$$

Where,

V_A, V_B, V_C, V_D : Real-time velocities of the four Mecanum wheels,

V_x : Velocity of the Mecanum wheel in the X-axis direction

2. When the robot translates along the Y-axis, the speed component of each wheel can be calculated using the following formula:

$$V_A = -V_Y \quad , \quad V_B = +V_Y \quad , \quad V_C = -V_Y \quad , \quad V_D = +V_Y$$

Where, V_y is the velocity of the robot in the Y-axis direction.

3. When the robot rotates along the Z-axis, the speed component of each wheel can be calculated using the following formulas:

$$V_A = -V_\omega * \left(\frac{1}{2}L + \frac{1}{2}H \right) \quad , \quad V_B = V_\omega * \left(\frac{1}{2}L + \frac{1}{2}H \right)$$

$$V_C = -V_\omega * \left(\frac{1}{2}L + \frac{1}{2}H \right) \quad , \quad V_D = V_\omega * \left(\frac{1}{2}L + \frac{1}{2}H \right)$$

Where, $V_R = V_x + \frac{V_\omega * D}{2}$: The angular velocity of the Mecanum wheel chassis (i.e., the speed at which the chassis rotates around its own center)

4. Combining the velocities in the X, Y, and Z directions allows for the computation of the rotation speeds of the four wheels based on the motion state of the car.

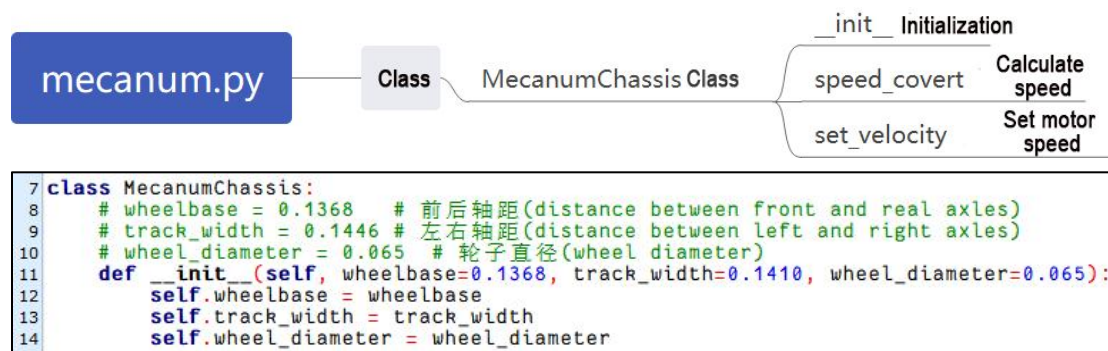
$$V_A = V_X - V_Y - V_\omega * \left(\frac{1}{2}L + \frac{1}{2}H \right) , \quad V_B = V_X + V_Y + V_\omega * \left(\frac{1}{2}L + \frac{1}{2}H \right)$$

$$V_C = V_X - V_Y - V_\omega * \left(\frac{1}{2}L + \frac{1}{2}H \right) , \quad V_D = V_X + V_Y + V_\omega * \left(\frac{1}{2}L + \frac{1}{2}H \right)$$

4. Program Outcome

The program file is saved in:

ros2_ws\src\driver\controller\controller\mecanum.py



Mecanum wheel kinematics class, used to calculate wheel speeds and implement Mecanum wheel kinematics.

Init:

```

11 def __init__(self, wheelbase=0.1368, track_width=0.1446, wheel_diameter=0.065):
12     self.wheelbase = wheelbase
13     self.track_width = track_width
14     self.wheel_diameter = wheel_diameter

```

Initialize the wheel dimensions for convenient subsequent calculations.

speed_covert:

```

16 def speed_covert(self, speed):
17     """
18     covert speed m/s to rps/s
19     :param speed:
20     :return:
21     """
22     # distance / circumference = rotations per second
23     return speed / (math.pi * self.wheel_diameter)

```

Convert m/s to rps/s based on the wheel's parameters.

set_velocity:

```

25 def set_velocity(self, linear_x, linear_y, angular_z):
26     """
27     Use polar coordinates to control moving
28     | | | x
29     v1 motor1| | motor3 v3
30     | + y - | |
31     v2 motor2| | motor4 v4
32     :param speed: m/s
33     :param direction: Moving direction 0~2pi, 1/2pi<--- | ---> 3/2pi
34     :param angular_rate: The speed at which the chassis rotates rad/sec
35     :param fake:
36     :return:
37     """
38     # vx = speed * math.sin(direction)
39     # vy = speed * math.cos(direction)
40     # vp = angular_rate * (self.wheelbase + self.track_width) / 2
41     # v1 = vx - vy - vp
42     # v2 = vx + vy - vp
43     # v3 = vx + vy + vp
44     # v4 = vx - vy + vp
45     # v_s = [self.speed_convert(v) for v in [v1, v2, -v3, -v4]]
46     motor1 = (linear_x - linear_y - angular_z * (self.wheelbase + self.track_width) / 2)
47     motor2 = (linear_x + linear_y - angular_z * (self.wheelbase + self.track_width) / 2)
48     motor3 = (linear_x + linear_y + angular_z * (self.wheelbase + self.track_width) / 2)
49     motor4 = (linear_x - linear_y + angular_z * (self.wheelbase + self.track_width) / 2)
50     v_s = [self.speed_convert(v) for v in [-motor1, -motor2, motor3, motor4]]
51     data = []
52     for i in range(len(v_s)):
53         msg = MotorState()
54         msg.id = i + 1
55         msg.rps = float(v_s[i])
56         data.append(msg)
57
58     msg = MotorsState()
59     msg.data = data
60     return msg

```

Based on the input speed parameters, decompose them, calculate the speeds using speed_convert, and then publish the calculated radian speeds to the motors.